



Flujo Operativo Personal para Proyectos Universitarios de Software

Integración de la Checklist Maestra v4, Sprint 0, UX Pilot/Figma,
Gentle AI ODD Briefing Generator, Pi y Organic Driven Development

Responsable

Alex Saúl Fernández Valdez

Complemento personal

Gentle AI ODD Briefing Generator

Documento metodológico y operativo

Basado en la Checklist Maestra Universal para Proyectos Universitarios de Software v4
y en una adaptación personal del flujo ODD de Gentleman para trabajo con Pi.

Octubre de 2026

Índice

1. Propósito del documento	3
2. Fundamentos que gobiernan mi forma de trabajar	3
3. Ecosistema de herramientas	4
3.1. Checklist Maestra Universal v4	4
3.2. UX Pilot en Figma	5
3.3. Gentle AI ODD Briefing Generator	5
3.4. Pi y Gentle Shell	6
4. Vista general de mi flujo operativo	7
5. Fase 1 — Documentación y comprensión mediante la checklist	7
5.1. Primero: comprender y demostrar el problema	7
5.2. De evidencia a decisiones	8
5.3. Baseline antes del desarrollo	8
6. Fase 2 — Sprint 0 como adaptación personal	9
6.1. Objetivo del Sprint 0	9
6.2. Contenido típico	9
7. Fase 3 — Diseño visual con UX Pilot en Figma	10
7.1. Qué espero obtener	10
7.2. Cómo se utilizan después	11
8. Fase 4 — Preparación de la HU	11
8.1. Definition of Ready personal	11
8.2. La Plantilla Maestra como interfaz entre HU y complemento	12
9. Fase 5 — Generación del ODD Execution Brief	13
9.1. Responsabilidad del complemento	13
9.2. Lo que no debe hacer	13
10. Fase 6 — Ejecución con Pi, Gentle Shell y ODD	14
10.1. ODD como flujo base de ejecución	14
10.2. Cómo encaja ODD dentro de mi flujo	14
10.2.1. Authorize	14
10.2.2. Explore	14
10.2.3. Classify	15
10.2.4. Track if substantial	15
10.2.5. Implement	15
10.2.6. Check	15
10.2.7. Close	15

11.Test-first y verificación en mi adaptación	16
12.Documentación permanente, evidencia y trazabilidad	16
12.1. Dos capas distintas	16
12.2. Regla de sincronización conceptual	16
13.Política Git y control humano	17
13.1. Maintainer Review	17
14.Matriz de responsabilidades	18
15.Trazabilidad extremo a extremo	18
16.Procedimiento operativo por cada HU	19
17.Cómo adapto el flujo base de Gentleman	19
18.Antipatronos que evito	20
19.Ejemplo resumido de aplicación	21
20.Regla maestra de calidad de mi flujo	22
21.Referencias	22
A. Checklist compacta del flujo completo	23

1. Propósito del documento

Este documento formaliza el flujo operativo que utilizo para desarrollar proyectos universitarios de software. El objetivo no es describir únicamente una metodología de programación, sino representar el proceso completo que sigo desde la comprensión del problema hasta la implementación incremental de historias de usuario, su verificación, la actualización documental y la entrega.

Mi flujo combina cuatro elementos principales:

1. la **Checklist Maestra Universal para Proyectos Universitarios de Software v4** como marco de análisis, documentación, trazabilidad y control de calidad;
2. un **Sprint 0** personal como transición entre el análisis/documentación y el inicio técnico del producto;
3. **UX Pilot dentro de Figma** para producir referencias visuales de las pantallas antes de implementar las historias de usuario;
4. el ecosistema **Gentle AI + Gentle Shell + Pi**, utilizando Organic Driven Development (ODD) y mi complemento **Gentle AI ODD Briefing Generator** para transformar cada HU preparada en un handoff de ejecución preciso.

Idea rectora

Mi adaptación mantiene una separación deliberada entre dos responsabilidades:

- **Antes de Pi:** comprender, documentar, validar, diseñar el alcance, preparar la arquitectura mínima, construir referencias UX y dejar la HU suficientemente definida.
- **Con Pi:** explorar el repositorio real, derivar el plan técnico desde evidencia, implementar mediante ODD, verificar y mantener la documentación/evidencia alineada.

Por tanto, ODD no sustituye mi proceso universitario completo: actúa como el flujo de ejecución de software dentro de un proceso mayor que yo adapté a mis necesidades.

2. Fundamentos que gobiernan mi forma de trabajar

La Checklist Maestra establece varios principios que adopto como reglas transversales del proyecto:

- **Comprender antes de construir:** primero se demuestra y entiende el problema; después se diseña la solución.
- **Evidencia real:** entrevistas, encuestas, observaciones, métricas, validaciones y pruebas no se presentan como reales si no fueron ejecutadas.
- **Trazabilidad:** una decisión importante debe poder rastrearse hacia una necesidad, requisito, regla, HU, diseño, implementación, prueba o evidencia.

- **Fuente editable:** diagramas y artefactos visuales mantenibles deben conservar tanto su fuente como su render.
- **Mínima duplicación:** cada tipo de información debe poseer una fuente principal para evitar copias divergentes.
- **Evolución controlada:** cuando cambia comportamiento, alcance, arquitectura, datos o estados, deben revisarse también los documentos, diagramas, pruebas y trazabilidad afectados.

Estos principios condicionan todo el flujo posterior, incluido el trabajo con IA. La IA no reemplaza la evidencia ni la validación humana; se utiliza para estructurar, investigar, implementar y verificar de manera controlada.

3. Ecosistema de herramientas

3.1. Checklist Maestra Universal v4

Es el marco de cobertura del proyecto. La utilizo para evitar comenzar a programar sin haber definido adecuadamente:

- contexto y problema;
- stakeholders;
- evidencia y relevamiento;
- hallazgos y necesidades;
- objetivos;
- alcance y MVP;
- SRS, RF, RNF y reglas del dominio;
- UX y flujos;
- backlog, épicas e historias;
- criterios de aceptación y Definition of Ready;
- modelado, arquitectura y datos;
- seguridad, riesgos y pruebas;
- plan de implementación;
- seguimiento, trazabilidad, validación y evidencias.

3.2. UX Pilot en Figma

UX Pilot lo incorporo como herramienta de **preparación visual**. Una vez que el problema, requisitos, alcance y base técnica están suficientemente establecidos, lo utilizo dentro de Figma para generar referencias de las pantallas.

Estas referencias sirven para:

- traducir requisitos y flujos a una intención visual concreta;
- visualizar pantallas clave antes de implementarlas;
- reducir ambigüedad en estados, jerarquía visual y responsive;
- alimentar el contexto de la HU;
- permitir que Pi tenga una referencia visual explícita durante la implementación.

Una referencia de UX Pilot/Figma es una **fuentes de intención visual**, no una autorización automática para ampliar el alcance funcional. Si una imagen muestra elementos decorativos, datos ficticios o funciones no aprobadas, esos elementos no se convierten por sí solos en requisitos.

3.3. Gentle AI ODD Briefing Generator

Mi complemento personal actúa como un **traductor entre la HU preparada y la ejecución con Pi**. Su trabajo no es programar.

A partir de la HU, de la documentación permanente y de la **Plantilla Maestra de Prompt — Gentle AI ODD Briefing Generator**, el complemento organiza un único **ODD Execution Brief**.

El briefing debe transmitir, cuando corresponda:

- objetivo y problema;
- alcance autorizado y non-goals;
- fuentes de verdad;
- comportamiento actual y comportamiento esperado;
- reglas de negocio e invariantes;
- criterios de aceptación y edge cases;
- contexto técnico conocido, sin inventar implementación;

- contratos, datos, estados y compatibilidad;
- expectativas UX;
- decisiones ya aprobadas;
- incertidumbres materiales;
- contexto de pruebas y evidencia esperada;
- documentación permanente afectada;
- permisos Git/delivery;
- condición de parada y Maintainer Review;
- handoff de ejecución para Pi.

3.4. Pi y Gentle Shell

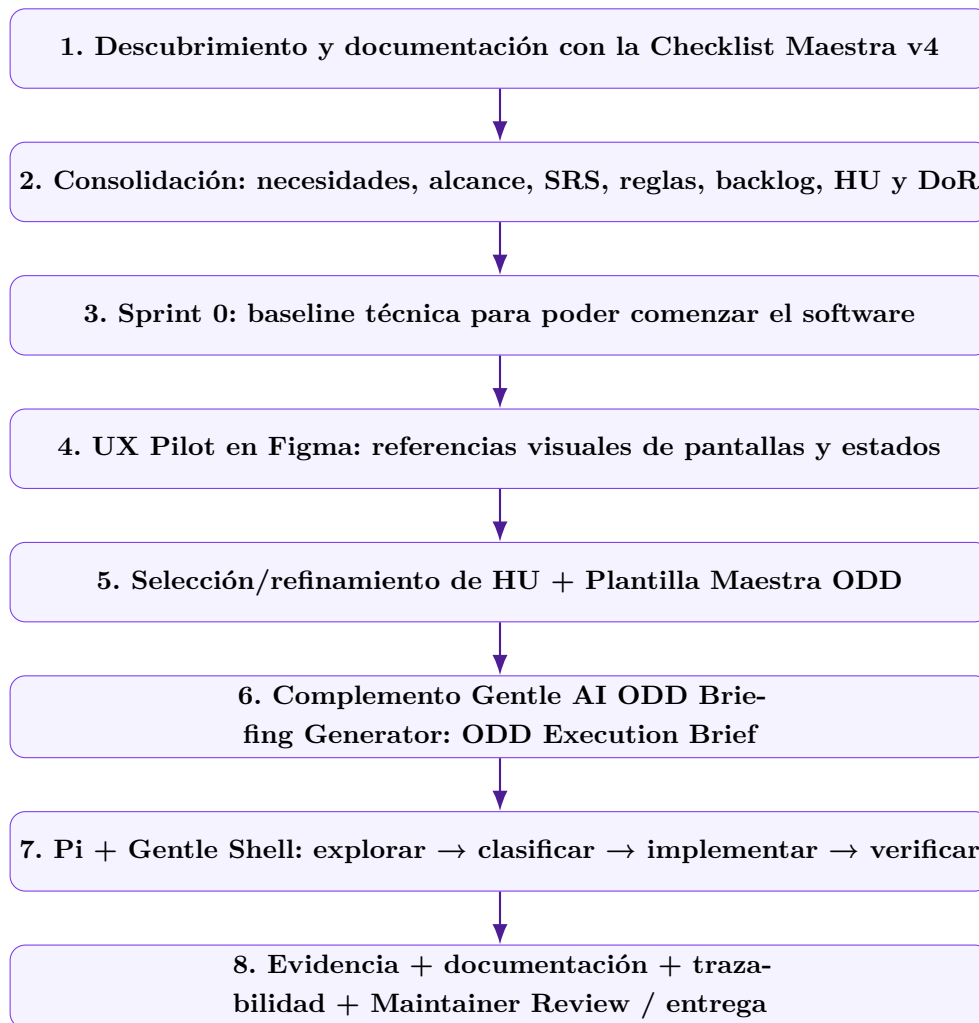
Pi es el agente con el que ejecuto el trabajo sobre el repositorio. Gentle Shell proporciona el entorno/orquestación específico de Pi y aplica el flujo ODD, delegación, memoria y controles correspondientes.

En mi modelo:

- **el complemento conoce la intención;**
- **Pi conoce la realidad del repositorio después de explorarlo;**
- **yo conservo la autoridad sobre alcance y decisiones de producto.**

Esta separación es fundamental para evitar que el briefing invente archivos, clases, endpoints, runners o una arquitectura que todavía no fue comprobada en el código.

4. Vista general de mi flujo operativo



Lectura correcta del diagrama

El flujo no es una cascada rígida. Existe retroalimentación. Si una HU revela una regla faltante, si la validación contradice una hipótesis o si la implementación descubre una limitación real, vuelvo al artefacto permanente correspondiente y actualizo la trazabilidad. Lo que evito es comenzar cada cambio desde cero o permitir que una decisión descubierta en código reescriba silenciosamente el alcance.

5. Fase 1 — Documentación y comprensión mediante la checklist

5.1. Primero: comprender y demostrar el problema

Antes de escribir software, utilizo técnicas de relevamiento reales según el contexto del proyecto:

- entrevistas;
- encuestas o cuestionarios;

- observación;
- análisis documental;
- análisis de procesos;
- análisis de datos existentes;
- benchmarking;
- talleres o focus groups cuando aportan valor;
- revisión bibliográfica;
- pruebas de concepto o prototipado cuando la incertidumbre lo requiere.

Cada técnica se justifica por la pregunta que debe responder. El objetivo no es “tener una entrevista” o “tener una encuesta”, sino producir evidencia útil para decidir.

5.2. De evidencia a decisiones

La cadena documental que busco es:

**Evidencia → Hallazgo → Necesidad → Objetivo → Alcance/MVP → Requisito o regla
→ HU**

Esta cadena reduce el riesgo de crear funcionalidades que no responden a necesidades reales.

5.3. Baseline antes del desarrollo

Antes de considerar una HU lista para ejecución, procuro que existan como mínimo, en el nivel de profundidad que el proyecto requiera:

- problema entendido y sustentado;
- stakeholders y actores identificados;
- alcance y fuera de alcance;
- MVP delimitado;
- SRS mínimo;
- RF/RNF y reglas principales;
- backlog priorizado;
- criterios de aceptación;
- Definition of Ready;

- arquitectura mínima y datos suficientemente definidos;
- estrategia inicial de pruebas;
- riesgos y dependencias relevantes.

6. Fase 2 — Sprint 0 como adaptación personal

La checklist no necesita llamar literalmente “Sprint 0” a esta preparación. Yo utilizo ese nombre para agrupar el trabajo técnico que convierte la documentación en una base de software ejecutable.

6.1. Objetivo del Sprint 0

El Sprint 0 responde a la pregunta:

¿Está el proyecto preparado técnica y organizativamente para que la primera HU pueda implementarse sin rehacer el análisis principal ni improvisar la base del repositorio?

6.2. Contenido típico

Dependiendo del proyecto, mi Sprint 0 puede incluir:

- creación y organización del repositorio;
- README y convenciones;
- estrategia de ramas;
- estructura documental;
- elección y configuración inicial del stack;
- instalación de dependencias;
- variables/configuración de entorno sin exponer secretos;
- skeleton de frontend y backend cuando corresponda;
- arquitectura mínima viable;
- base del modelo de datos;
- autenticación/autorización base si es prerequisite transversal;
- estrategia de migraciones/seeds cuando aplique;
- configuración de tests/runners;
- automatizaciones básicas de calidad;

- CI inicial cuando aporta valor;
- Docker/ambientes si el proyecto los requiere;
- tablero y primera iteración priorizada;
- verificación de que la primera HU puede comenzar con dependencias críticas resueltas.

Sprint 0 no es “implementar todo lo técnico primero”

El Sprint 0 debe ser suficiente para habilitar el desarrollo incremental, no una excusa para construir una plataforma completa antes de entregar valor. La arquitectura se prepara hasta el nivel necesario para que el desarrollo por HUs sea viable y coherente.

7. Fase 3 — Diseño visual con UX Pilot en Figma

Después del Sprint 0 preparo la referencia visual de las pantallas.

El orden es deliberado: UX Pilot recibe un proyecto que ya tiene problema, alcance, reglas, actores y una base técnica suficientemente comprendida. De esa manera, el diseño visual se apoya en decisiones previamente justificadas.

7.1. Qué espero obtener

Para las pantallas relevantes procuro cubrir:

- flujo principal;
- pantallas clave;
- navegación;
- estados de carga;
- estados vacíos;
- errores;
- confirmaciones;
- feedback al usuario;
- responsive cuando corresponda;
- accesibilidad básica;
- consistencia visual entre pantallas relacionadas.

7.2. Cómo se utilizan después

Las capturas/referencias de Figma se incorporan como contexto de la HU. El briefing debe declarar explícitamente qué partes son:

- comportamiento funcional obligatorio;
- referencia visual;
- dato de ejemplo;
- elemento fuera de alcance.

Así evito que Pi interprete una maqueta como si fuera una especificación funcional completa.

8. Fase 4 — Preparación de la HU

Una HU no pasa directamente del backlog a Pi. Primero verifico que esté suficientemente refinada.

8.1. Definition of Ready personal

Además del DoR del proyecto, para este flujo considero especialmente importante que la HU tenga:

- ID y épica;
- rol, acción y beneficio;
- necesidad/requisito relacionado;
- prioridad;
- dependencias;
- criterios de aceptación;
- reglas de negocio aplicables;
- datos/entidades relevantes;
- permisos;
- precondiciones;
- alternativas y errores importantes;
- riesgos;
- referencia UX/Figma si existe una pantalla asociada;

- claridad suficiente para que el complemento pueda construir el handoff.

8.2. La Plantilla Maestra como interfaz entre HU y complemento

Con la HU preparada, utilizo la **Plantilla Maestra de Prompt — Gentle AI ODD Briefing Generator**.

La plantilla no es un plan técnico del repositorio. Es una estructura para suministrar al complemento el contexto que conozco y las decisiones que ya están autorizadas.

Los grupos de información más importantes son:

- proyecto e identificador del cambio;
- objetivo;
- alcance y non-goals;
- sources of truth;
- comportamiento actual y esperado;
- reglas y aceptación;
- edge cases;
- arquitectura y stack conocidos;
- archivos/símbolos únicamente cuando están realmente confirmados;
- contratos/datos;
- estados y concurrencia;
- migración/data safety;
- seguridad/permisos;
- UX y referencias visuales;
- decisiones aprobadas;
- material uncertainty;
- contexto de test/verificación;
- documentación/evidencia;
- Git/delivery authorization;
- Maintainer Review.

Regla esencial

La plantilla organiza lo que **sé y decido antes de ejecutar**. Pi debe determinar lo que **sólo puede saberse inspeccionando el repositorio real**.

9. Fase 5 — Generación del ODD Execution Brief

El complemento recibe la plantilla completada y produce un único **ODD Execution Brief**.

9.1. Responsabilidad del complemento

El complemento debe:

- normalizar información redundante;
- preservar decisiones exactas;
- separar hechos, decisiones, incertidumbres y responsabilidades de ejecución;
- detectar contradicciones;
- evitar inventar detalles de implementación;
- explicitar fuentes de verdad;
- fijar criterios de aceptación;
- transferir las restricciones y permisos;
- producir un handoff específico para Pi.

9.2. Lo que no debe hacer

No debe:

- programar;
- afirmar que inspeccionó el repositorio si no lo hizo;
- inventar rutas, clases, endpoints o tablas;
- inventar comandos;
- anticipar resultados PASS;
- fijar una descomposición técnica rígida antes de explorar;
- autorizar scope adicional;

- asumir permisos Git no proporcionados.

10. Fase 6 — Ejecución con Pi, Gentle Shell y ODD

10.1. ODD como flujo base de ejecución

El flujo de Organic Driven Development de Gentleman sirve como base del trabajo con el agente. La explicación audiovisual de ODD que utilizo como referencia es:

ODD — Organic Driven Development (video de referencia)

Como referencia adicional del flujo base de Gentle AI:

Flujo de trabajo base de Gentle AI (video de referencia)

Mi proceso **no copia de forma literal** esos materiales. Los adapto a mi contexto universitario y a mi disciplina documental.

10.2. Cómo encaja ODD dentro de mi flujo

El handoff a Pi inicia una ejecución conceptual como:

Authorize → Explore → Resolve material uncertainty → Classify → Track if substantial → Implement → Check → Close

10.2.1. Authorize

Pi debe respetar el alcance y los permisos del briefing. Encontrar un problema adyacente no autoriza automáticamente corregirlo.

10.2.2. Explore

Pi inspecciona el repositorio actual para descubrir:

- archivos y símbolos reales;
- patrones existentes;
- contratos;
- arquitectura efectiva;
- tests/runners;
- migraciones necesarias;
- discrepancias entre documentación y código.

10.2.3. Classify

Pi determina si el cambio es pequeño o sustancial.

- trabajo pequeño y entendido: ejecución ligera;
- trabajo sustancial: progreso recuperable mediante tracking ODD.

10.2.4. Track if substantial

Si el trabajo es sustancial, el runtime puede crear un documento del tipo:

`odd/tasks/<feature-name>.md`

y mantener un mirror project-scoped en Engram cuando esté disponible.

Este documento **no reemplaza** la HU, el SRS ni las reglas del dominio. Es un ledger operacional de implementación.

10.2.5. Implement

Pi deriva work units desde el repositorio real. La HU sigue siendo la unidad de valor; los work units son unidades técnicas de ejecución.

10.2.6. Check

Los resultados deben demostrarse con evidencia observada: tests, build, typecheck, lint, integración, E2E, validación visual, API, datos, logs u otros checks aplicables.

10.2.7. Close

Cerrar no significa publicar automáticamente. El cierre debe declarar:

- qué se implementó;
- qué evidencia existe;
- qué no pudo verificarse;
- qué quedó fuera de alcance;
- qué documentación se actualizó;
- cuál es el estado Git;
- cuál es el siguiente límite humano o de delivery.

11. Test-first y verificación en mi adaptación

Para cambios de comportamiento, si existe un test significativo, ejecutable, determinista y con resultado esperado claro, favorezco:

RED → GREEN → REFACTOR

Sin embargo, no fuerzo un RED artificial cuando:

- el runner no está disponible;
- el cambio es documental;
- el comportamiento no se representa de manera útil con un test determinista;
- el entorno falla antes de probar la funcionalidad;
- el costo de construir el test no es proporcional al cambio.

En esos casos se requiere verificación funcional o estructural proporcional, dejando constancia honesta de lo ejecutado.

12. Documentación permanente, evidencia y trazabilidad

Una característica central de mi adaptación es que ODD no reemplaza la documentación universitaria.

12.1. Dos capas distintas

Capa	Propósito
Documentación permanente	Describe la verdad vigente del proyecto: problema, SRS, requisitos, reglas, HUs, arquitectura, ADR, datos, UX, pruebas, trazabilidad y evidencias.
Tracking ODD operacional	Permite recuperar el estado de implementación de una feature sustancial: tareas, progreso, evidencia observada y siguiente acción.

12.2. Regla de sincronización conceptual

Cuando una HU modifica una verdad permanente del sistema, el trabajo no está completamente terminado si la documentación conocida queda desactualizada.

Por ello, al final de cada HU reviso:

- requisitos afectados;
- reglas;
- diagramas;
- arquitectura;
- modelo de datos;
- API/contratos;
- UX;
- trazabilidad;
- evidencia;
- documentación de sprint.

13. Política Git y control humano

Mi flujo distingue explícitamente autorización de implementación y autorización de delivery.

Operación	Regla
Explorar repositorio	Permitido cuando el trabajo requiere entender la implementación.
Modificar código	Sólo dentro del alcance autorizado de la HU/cambio.
Commit local	Debe estar explícitamente autorizado cuando corresponda.
Push	No se deduce de permiso de commit.
Crear PR	Decisión separada.
Merge	Decisión separada.
Rebase / force push	Requiere política o autorización explícita; force push no se asume.
Deployment	Siempre se trata como decisión de delivery separada.

13.1. Maintainer Review

Para cambios importantes suelo utilizar un límite humano:

IMPLEMENTATION + APPLICABLE CHECKS COMPLETE



PENDING MAINTAINER REVIEW

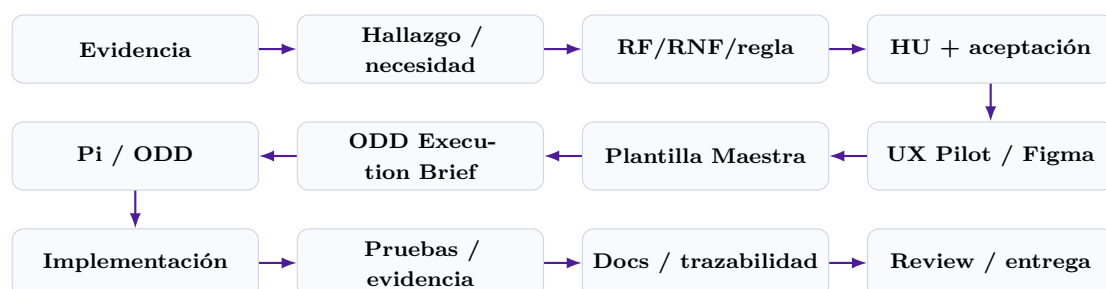
En ese punto reviso el estado antes de permitir la publicación o integración cuando corresponda.

14. Matriz de responsabilidades

Actor / herramienta	Responsabilidad principal	No debe sustituir
Alex / responsable	Decisiones de producto, alcance, prioridad, validación, riesgo aceptado y delivery.	La evidencia real ni la verificación técnica.
Checklist Maestra v4	Cobertura metodológica, documental, académica y de calidad.	El juicio contextual sobre qué elementos aplican al proyecto.
Entrevistas, encuestas, observación, análisis	Producir evidencia sobre problema, necesidades y contexto.	Requisitos ficticios ni validaciones simuladas.
Sprint 0	Preparar la baseline técnica y organizativa para desarrollar.	La entrega incremental de valor durante los sprints posteriores.
UX Pilot / Figma	Producir referencias visuales y apoyar la definición de pantallas/estados.	Las reglas de negocio ni el alcance funcional aprobado.
Plantilla Maestra	Estructurar la información de la HU antes del complemento.	La exploración del repositorio.
Gentle AI ODD Briefing Generador	Generar el ODD Execution Brief y preservar intención, aceptación y límites.	La implementación técnica real.
Pi + Gentle Shell	Explorar, planificar desde evidencia, implementar, delegar y verificar.	La autoridad humana sobre scope y delivery.
ODD tracking / Engram	Dar continuidad recuperable al trabajo sustancial.	La documentación permanente del proyecto.

15. Trazabilidad extremo a extremo

La trazabilidad deseada puede representarse de esta forma:



Esta cadena permite responder no sólo “qué se programó”, sino **por qué existe, qué necesidad satisface, cómo se diseñó, cómo se comprobó y qué evidencia respalda el resultado.**

16. Procedimiento operativo por cada HU

Cuando el proyecto ya superó las etapas de análisis, Sprint 0 y preparación UX, repito el siguiente microciclo:

1. Seleccionar la HU priorizada.
2. Confirmar que sigue cumpliendo Definition of Ready.
3. Revisar decisiones, reglas, dependencias y referencias visuales.
4. Completar la Plantilla Maestra con el contexto específico.
5. Enviar la plantilla al complemento.
6. Revisar el ODD Execution Brief generado.
7. Entregar el briefing a Pi en el repositorio correcto.
8. Permitir que Pi explore antes de fijar archivos/tasks/commands.
9. Resolver únicamente decisiones humanas realmente bloqueantes.
10. Implementar con work units derivados del repositorio.
11. Aplicar test-first cuando sea significativo y ejecutable.
12. Ejecutar checks proporcionales.
13. Generar/organizar evidencias.
14. Actualizar documentación permanente y trazabilidad.
15. Llegar a la condición de revisión definida.
16. Autorizar, si corresponde, push/PR/merge/deployment por separado.
17. Pasar a la siguiente HU o registrar cambios en backlog.

17. Cómo adapto el flujo base de Gentleman

Mi adaptación agrega una capa previa y una capa documental posterior alrededor de ODD.

Aspecto	Flujo base ODD / Gentle	Mi adaptación personal
Inicio	Describir un outcome autorizado y explorar el código.	Antes del outcome existe una baseline de problema, evidencia, necesidades, requisitos, backlog y HU.
Diseño inicial	Se deriva del contexto y del repositorio.	Añado Sprint 0 para establecer una base técnica mínima antes de iniciar las HUs.
UX	Depende del proyecto/contexto.	Genero referencias de pantallas con UX Pilot en Figma antes de implementar HUs visuales.
Handoff	Petición al agente.	Uso una Plantilla Maestra y un complemento que transforma la HU en un ODD Execution Brief.
Plan técnico	El agente explora y deriva el trabajo.	Mantengo este principio: el complemento no inventa el plan técnico.
Tracking	Sólo cuando el trabajo es sustancial.	Lo mantengo separado de HUs/SRS; ODD tracking es operacional.
Verificación	Checks aplicables y evidencia observada.	La conecto explícitamente con DoD, documentación, evidencia académica y trazabilidad.
Cierre	Resultado técnico y siguiente acción.	Puedo añadir Maintainer Review antes de cualquier delivery remoto.

18. Antipatrones que evito

- Programar antes de comprender el problema.
- Inventar entrevistas, encuestas, usuarios, resultados o validaciones.
- Convertir cada sugerencia de un stakeholder en un requisito.
- Diseñar pantallas sin reglas o flujo suficientemente claros.
- Usar una imagen de Figma como contrato funcional completo.

- Pedir al complemento que invente el repositorio.
- Predefinir tareas técnicas sin inspeccionar código real.
- Permitir que Pi expanda alcance por iniciativa propia.
- Confundir tracking ODD con documentación permanente.
- Marcar una HU Done con documentación conocida como desactualizada.
- Tratar un fallo ambiental como fallo funcional sin evidencia.
- Considerar commit, push, PR, merge y deployment como una única autorización.
- Utilizar tests sólo para acumular cantidad sin demostrar comportamiento.
- Mantener diagramas/renders desincronizados de sus fuentes editables.

19. Ejemplo resumido de aplicación

Supongamos una HU de importación CSV.

1. Durante relevamiento se descubre que el registro manual masivo consume tiempo y produce errores.
2. El hallazgo se convierte en necesidad y luego en requisito.
3. El backlog crea una HU de importación.
4. Sprint 0 ya dejó frontend, backend, persistencia y tests base disponibles.
5. UX Pilot genera la referencia de selección de archivo, preview, errores y resultado.
6. La HU se completa con reglas de campos, duplicados, permisos y atomicidad.
7. La Plantilla Maestra especifica scope, non-goals, sources of truth, acceptance y evidencia.
8. El complemento genera el ODD Execution Brief.
9. Pi explora el repositorio para localizar validadores, patrones de upload, API, transacciones y test runners reales.
10. Si el trabajo es sustancial, crea tracking ODD.
11. Implementa por work units y verifica.
12. Se producen evidencias y se actualiza la HU, requisitos, API/arquitectura si corresponde y trazabilidad.
13. Se detiene en Maintainer Review si esa es la política configurada.

Lo importante es que ninguna capa trabaja aislada: cada paso consume decisiones previas y produce evidencia o conocimiento para el siguiente.

20. Regla maestra de calidad de mi flujo

Antes de considerar que una HU o un incremento está realmente terminado, debo poder responder:

- ¿Qué problema o necesidad justifica el cambio?
- ¿Qué fuente respalda esa necesidad?
- ¿Qué requisito/regla/HU autoriza la implementación?
- ¿Qué parte estaba dentro y fuera del alcance?
- ¿Qué referencia UX debía respetarse?
- ¿Qué descubrió Pi al explorar el repositorio?
- ¿Qué se implementó realmente?
- ¿Qué checks se ejecutaron y con qué resultado?
- ¿Qué no pudo verificarse y por qué?
- ¿Qué evidencia existe?
- ¿Qué documentación quedó actualizada?
- ¿Cuál es el estado Git real?
- ¿Qué decisión humana falta, si existe?

Si estas preguntas no pueden responderse con claridad, el flujo todavía tiene una ruptura de trazabilidad o de evidencia.

21. Referencias

Fuentes propias del flujo

- **Checklist Maestra Universal para Proyectos Universitarios de Software v4.** Base metodológica y documental utilizada por el responsable.
- **Plantilla Maestra de Prompt — Gentle AI ODD Briefing Generator.** Interfaz de preparación de contexto por HU.
- **Gentle AI ODD Briefing Generator.** Complemento personal utilizado para construir el handoff para Pi.

Referencias de Gentleman

- Organic Driven Development (ODD):
<https://www.youtube.com/watch?v=WM2JAuR-MOU>
- Flujo de trabajo base de Gentle AI:
<https://www.youtube.com/watch?v=-8neJsLJsXo&t=3036s>
- [Gentle AI — Intended Usage](#)
- [Gentle AI — Pi integration](#)
- [Gentle Shell](#)

Alcance de estas referencias

Los videos y la documentación de Gentleman explican el flujo base de ODD/Gentle AI/Gentle Shell. El proceso descrito en este documento es una **adaptación personal de Alex Saúl Fernández Valdez**: incorpora la Checklist Maestra, un Sprint 0 propio, UX Pilot/Figma, la Plantilla Maestra y el complemento de briefings antes de la ejecución con Pi.

A. Checklist compacta del flujo completo

- ☐ Revisar consigna/rúbrica y gobierno del proyecto.
- ☐ Comprender situación actual y problema.
- ☐ Obtener evidencia real.
- ☐ Consolidar hallazgos y necesidades.
- ☐ Definir objetivos, alcance y MVP.
- ☐ Crear SRS, RF, RNF y reglas.
- ☐ Crear backlog, épicas, HUs y aceptación.
- ☐ Alcanzar DoR suficiente.
- ☐ Completar Sprint 0 técnico.
- ☐ Generar referencias UX Pilot/Figma.
- ☐ Seleccionar HU priorizada.
- ☐ Completar Plantilla Maestra.
- ☐ Generar ODD Execution Brief con el complemento.

- ☐ Entregar briefing a Pi/Gentle Shell.
- ☐ Explorar repositorio y resolver incertidumbres reales.
- ☐ Clasificar small/substantial.
- ☐ Crear tracking ODD sólo si corresponde.
- ☐ Implementar por work units.
- ☐ Ejecutar test-first cuando aplique.
- ☐ Ejecutar verificación proporcional.
- ☐ Organizar evidencia.
- ☐ Actualizar documentación/trazabilidad.
- ☐ Llegar a Maintainer Review si está configurado.
- ☐ Autorizar delivery remoto por separado.
- ☐ Registrar review/retrospectiva y continuar el backlog.

Responsable: Alex Saúl Fernández Valdez

Documento de flujo operativo personal para proyectos universitarios de software.